MC-UES CI/CD 完整架構規格

文件性質：AI 輸出治理管道完整實作規格 語言：Go 1.22+ 適用場景：大型 AI 協作專案的語義治理

原始作者欄位

中文： 原創者：ChenTing（陳霆） 創辦職位：BearNetworkChain 創辦人、執行長、技術長 學術單位：東海大學管理學院 聯絡信箱：bnkt@bearnetwork.net Canonical DOI: doi:10.5281/zenodo.21009888

English: Author: ChenTing Title: Founder, CEO, and Chief Technology Officer, BearNetworkChain Affiliation: College of Management, Tunghai University Email: bnkt@bearnetwork.net Canonical DOI: doi:10.5281/zenodo.21009888

目錄

1. 架構總覽

1.1 設計哲學

MC-UES CI/CD 不是 RAG，不是多模型系統。它是一個**外部確定性驗證管道**，用於約束 AI 輸出，使其符合 MC-UES 規格。

```
AI 產生輸出（非確定性）
        ↓
[Gate 1] 結構完整性檢查（確定性）
        ↓ 通過
[Gate 2] 術語邊界檢查（確定性）
        ↓ 通過
[Gate 3] 紅旗偵測（確定性）
        ↓ 通過
[Gate 4] 決策日誌一致性檢查（確定性）
        ↓ 通過
[Gate 5] 人工核查點（關鍵決策才觸發）
        ↓ 通過
輸出寫入專案 → 蒸餾為狀態快照
```

1.2 核心原則

- **確定性**：相同輸入永遠產生相同驗證結果
- **外部性**：驗證邏輯不依賴 AI 自身判斷
- **可審計**：所有判定結果附帶完整追溯鏈
- **漸進式**：每個 Gate 獨立，可單獨啟用或停用
- **語義錨定**：術語表在每次對話重新注入，防止漂移

1.3 與傳統 CI/CD 的對應關係

傳統 CI/CD	MC-UES CI/CD
程式碼提交	AI 輸出提交
unit test	判定式驗證（Gate 1-4）
lint rules	術語邊界 + 紅旗偵測
type checker	結構完整性驗證
test coverage	證據充分性檢查
merge gate	語義審計門檻（Gate 5）
PR review	人工核查點
commit log	決策日誌（ADR）
branch protection	核心約束鎖定

---

2. 專案目錄結構

```
mc-ues-ci/
├── cmd/
│   └── gate/
│       └── main.go                 # CLI 主入口
├── internal/
│   ├── types/
│   │   └── types.go                # 共用型別定義
│   ├── gate1_structure.go          # Gate 1：結構完整性
│   ├── gate2_terminology.go        # Gate 2：術語邊界
│   ├── gate3_redflags.go           # Gate 3：紅旗偵測
│   ├── gate4_decisions.go          # Gate 4：決策日誌一致性
│   ├── gate5_checkpoint.go         # Gate 5：人工核查點
│   ├── report.go                   # 審計報告產生
│   ├── distill.go                  # 蒸餾協議
│   ├── snapshot.go                 # 狀態快照管理
│   └── loader.go                   # 設定檔載入
├── config/
│   ├── terms.yaml                  # 術語表
│   └── gates.yaml                  # 門檻設定
├── decisions/
│   └── ADR-001.yaml                # 架構決策紀錄範例
├── snapshots/
│   └── .gitkeep
├── reports/
│   └── .gitkeep
├── .vscode/
│   ├── tasks.json                  # VSCode 任務整合
│   └── extensions.json             # 建議擴充套件
├── .github/
│   └── workflows/
│       └── mc-ues-gate.yml         # GitHub Actions CI
├── hooks/
│   └── pre-commit                  # Git pre-commit hook
├── go.mod
├── go.sum
└── Makefile
```

3. 設定檔規格

3.1 術語表 `config/terms.yaml`

```yaml
# MC-UES 術語表 v1.0
# 每次對話開始時注入語義層
version: "1.0"
updated: "2024-01-01"

terms:
  需求:
    definition: "可驗證的工程目標描述，必須包含驗證條件"
    forbidden_synonyms:
      - "希望"
      - "期望"
      - "最好能"
      - "理想上"
    required_evidence:
      - "驗證條件"
      - "判定標準"
    english_alias: "Requirement"

  約束:
    definition: "對需求、架構、行為、執行或證據所施加的正式限制"
    forbidden_synonyms:
      - "限制一下"
      - "盡量"
      - "儘可能"
    required_evidence:
      - "約束來源"
      - "違反條件"
    english_alias: "Constraint"

  架構:
    definition: "工程結構及其關係集合，必須可被形式化描述"
    forbidden_synonyms:
      - "大概的設計"
      - "差不多的結構"
    required_evidence:
      - "節點定義"
      - "關係定義"
    english_alias: "Architecture"

  證據:
    definition: "支持驗證結論之可重播材料，必須可觀測、可追溯"
    forbidden_synonyms:
      - "感覺上"
```

```yaml
      - "經驗告訴我"
      - "通常來說"
    required_evidence:
      - "唯一識別"
      - "來源標記"
      - "時間標記"
    english_alias: "Evidence"

  判定式:
    definition: "可計算的真值函數，輸出只能是真或假"
    forbidden_synonyms:
      - "大概是真的"
      - "應該成立"
    required_evidence:
      - "輸入域"
      - "輸出域"
      - "前提條件"
    english_alias: "Predicate"

  語義偏移:
    definition: "術語在對話過程中未授權改變定義的現象"
    forbidden_synonyms: []
    required_evidence:
      - "原始定義"
      - "偏移位置"
    english_alias: "Semantic Drift"

  幻覺:
    definition: "AI 生成無法追溯到可驗證來源的事實性聲明"
    forbidden_synonyms: []
    required_evidence:
      - "聲明內容"
      - "缺失的來源"
    english_alias: "Hallucination"

# 禁用詞全域清單（跨所有術語）
global_forbidden_phrases:
  - phrase: "大概"
    severity: "HIGH"
    mc_ues_ref: "12.3"
    reason: "非決定性推理，禁止作為判定依據"

  - phrase: "應該可以"
    severity: "HIGH"
    mc_ues_ref: "12.3"
    reason: "非決定性推理，禁止作為判定依據"
```

```yaml
- phrase: "看起來合理"
  severity: "HIGH"
  mc_ues_ref: "12.3"
  reason: "非決定性推理，禁止作為判定依據"

- phrase: "probably"
  severity: "HIGH"
  mc_ues_ref: "12.3"
  reason: "Non-deterministic reasoning"

- phrase: "should work"
  severity: "HIGH"
  mc_ues_ref: "12.3"
  reason: "Non-deterministic reasoning"

- phrase: "seems reasonable"
  severity: "HIGH"
  mc_ues_ref: "12.3"
  reason: "Non-deterministic reasoning"

- phrase: "通常來說"
  severity: "MEDIUM"
  mc_ues_ref: "4.7"
  reason: "隱含假設，需明示來源"

- phrase: "一般而言"
  severity: "MEDIUM"
  mc_ues_ref: "4.7"
  reason: "隱含假設，需明示來源"

- phrase: "眾所周知"
  severity: "HIGH"
  mc_ues_ref: "12.2"
  reason: "以印象替代形式證明"
```

3.2 門檻設定 `config/gates.yaml`

```yaml
# MC-UES Gate 門檻設定 v1.0
version: "1.0"

gates:
  gate1_structure:
    enabled: true
    required_fields:
      - field: "intent_summary"
        description: "意圖摘要"
        mc_ues_ref: "14.2"
      - field: "assumptions"
        description: "假設清單"
        mc_ues_ref: "14.4"
        allow_empty: false
      - field: "uncertainty"
        description: "不確定性聲明"
        mc_ues_ref: "14.5"
        allow_empty: false
      - field: "source_markers"
        description: "來源標記"
        mc_ues_ref: "公理四"
      - field: "predicate_results"
        description: "判定結果"
        mc_ues_ref: "14.2"
      - field: "red_flags"
        description: "紅旗清單"
        mc_ues_ref: "8.3"
      - field: "audit_conclusion"
        description: "審計結論"
        mc_ues_ref: "14.2"

  gate2_terminology:
    enabled: true
    terms_file: "config/terms.yaml"
    fail_on_forbidden_synonym: true
    fail_on_undefined_term: false  # 警告但不阻擋

  gate3_redflags:
    enabled: true
    fail_on_severity:
      - "HIGH"
    warn_on_severity:
      - "MEDIUM"
    max_medium_warnings: 3  # 超過此數量則升級為失敗
```

```yaml
  gate4_decisions:
    enabled: true
    decisions_dir: "decisions/"
    fail_on_conflict: true
    warn_on_pending: true

  gate5_checkpoint:
    enabled: true
    # 觸發人工核查點的條件
    triggers:
      - condition: "new_constraint_affects_confirmed_decision"
        description: "新約束影響已確認決策"
        action: "pause_and_notify"
      - condition: "new_exclusion_decision"
        description: "輸出包含新的排除決策"
        action: "pause_and_notify"
      - condition: "architecture_direction_change"
        description: "架構方向變更"
        action: "pause_and_notify"
      - condition: "high_red_flag_after_gate3"
        description: "Gate 3 出現 HIGH 紅旗後仍繼續"
        action: "pause_and_notify"
    # 自動通過條件
    auto_approve:
      - condition: "all_gates_pass_no_triggers"
        description: "所有 Gate 通過且無觸發條件"
        action: "auto_approve"

# 輸出格式設定
output:
  report_dir: "reports/"
  snapshot_dir: "snapshots/"
  format: "yaml"  # yaml 或 json
  include_timestamp: true
  include_version: true
```

3.3 架構決策紀錄範例 `decisions/ADR-001.yaml`

```yaml
# Architecture Decision Record
id: "ADR-001"
title: "資料庫選型"
status: "confirmed"        # confirmed / pending / superseded / deprecated
version: "1.0"
created: "2024-01-01"
updated: "2024-01-01"
superseded_by: null        # 若被取代，填入新 ADR id

context: |
  專案需要一個支援高寫入量且具備水平擴展能力的資料庫。

decision: |
  選用 PostgreSQL 作為主要資料庫。

rationale:
  - "支援 ACID 交易，符合資料一致性需求"
  - "成熟的水平擴展方案（Citus）"
  - "團隊已有操作經驗"

constraints:
  - "所有資料表必須有主鍵"
  - "不得使用儲存程序作為業務邏輯載體"
  - "查詢回應時間 P99 必須低於 100ms"

immutable_conditions:
  - "在未升版此 ADR 的情況下，不得更換資料庫引擎"
  - "在未新增 ADR 的情況下，不得引入第二個主要資料庫"

excluded_alternatives:
  - option: "MongoDB"
    reason: "資料結構已明確，文件型資料庫不提供額外價值"
  - option: "MySQL"
    reason: "JSON 支援較弱，不符合部分查詢需求"

evidence:
  - type: "benchmark"
    description: "PostgreSQL vs MySQL 寫入效能測試"
    location: "docs/benchmarks/db-comparison-2024.md"

tags:
  - "database"
  - "infrastructure"
  - "core"
```

4. Go 核心模組

4.1 `go.mod`

```
module github.com/yourorg/mc-ues-ci

go 1.22

require (
    gopkg.in/yaml.v3 v3.0.1
    github.com/spf13/cobra v1.8.0
    github.com/fatih/color v1.16.0
)
```

4.2 internal/types/types.go

```go
package types

import "time"

// GateResult 單一 Gate 的驗證結果
type GateResult struct {
    GateID      string        `yaml:"gate_id"`
    GateName    string        `yaml:"gate_name"`
    Status      GateStatus    `yaml:"status"`
    Timestamp   time.Time     `yaml:"timestamp"`
    Duration    int64         `yaml:"duration_ms"`
    Issues      []Issue       `yaml:"issues,omitempty"`
    Warnings    []Warning     `yaml:"warnings,omitempty"`
    MCUESRefs   []string      `yaml:"mc_ues_refs,omitempty"`
}

// GateStatus Gate 狀態
type GateStatus string

const (
    StatusPass            GateStatus = "PASS"
    StatusFail            GateStatus = "FAIL"
    StatusUnverified      GateStatus = "UNVERIFIED"
    StatusConditionalPass GateStatus = "CONDITIONAL_PASS"
    StatusConditionalFail GateStatus = "CONDITIONAL_FAIL"
    StatusPendingHuman    GateStatus = "PENDING_HUMAN"
)

// Issue 問題項目
type Issue struct {
    ID          string   `yaml:"id"`
    Type        string   `yaml:"type"`
    Severity    Severity `yaml:"severity"`
    Description string   `yaml:"description"`
    Location    string   `yaml:"location,omitempty"`
    MCUESRef    string   `yaml:"mc_ues_ref,omitempty"`
    RedFlag     string   `yaml:"red_flag,omitempty"`
}

// Warning 警告項目
type Warning struct {
    ID          string `yaml:"id"`
    Description string `yaml:"description"`
    Location    string `yaml:"location,omitempty"`
    MCUESRef    string `yaml:"mc_ues_ref,omitempty"`
```

```
}

// Severity 嚴重程度
type Severity string

const (
    SeverityHigh   Severity = "HIGH"
    SeverityMedium Severity = "MEDIUM"
    SeverityLow    Severity = "LOW"
)

// AuditReport 完整審計報告
type AuditReport struct {
    ReportID        string        `yaml:"report_id"`
    SpecVersion     string        `yaml:"spec_version"`
    Timestamp       time.Time     `yaml:"timestamp"`
    InputFile       string        `yaml:"input_file"`
    InputHash       string        `yaml:"input_hash"`
    GateResults     []GateResult  `yaml:"gate_results"`
    OverallStatus   GateStatus    `yaml:"overall_status"`
    RedFlags        []RedFlag     `yaml:"red_flags,omitempty"`
    AuditConclusion string        `yaml:"audit_conclusion"`
    TraceComplete   bool          `yaml:"trace_complete"`
    EvidenceSufficient bool       `yaml:"evidence_sufficient"`
}

// RedFlag 紅旗項目（對應 MC-UES Volume VII）
type RedFlag struct {
    Type        RedFlagType `yaml:"type"`
    Severity    Severity    `yaml:"severity"`
    Description string      `yaml:"description"`
    Location    string      `yaml:"location,omitempty"`
    MCUESRef    string      `yaml:"mc_ues_ref"`
}

// RedFlagType 紅旗類型（對應 MC-UES 8.2）
type RedFlagType string

const (
    RedFlagSemanticDrift          RedFlagType = "語義偏移"
    RedFlagRequirementMutation    RedFlagType = "需求變形"
    RedFlagArchitectureInconsist  RedFlagType = "架構不一致"
    RedFlagConstraintConflict     RedFlagType = "約束衝突"
    RedFlagEvidenceMissing        RedFlagType = "證據缺失"
    RedFlagHiddenAssumption       RedFlagType = "隱含假設"
    RedFlagVerificationFailure    RedFlagType = "驗證失敗"
```

```
    RedFlagPredicateViolation     RedFlagType = "判定式違反"
    RedFlagNonDeterministicReason RedFlagType = "非決定性推理"
    RedFlagHallucinatedDependency RedFlagType = "虛構依賴"
)

// AIOutput AI 輸出的結構化表示
type AIOutput struct {
    IntentSummary    string            `yaml:"intent_summary"`
    OntologyMapping  map[string]string `yaml:"ontology_mapping,omitempty"`
    Requirements     []string          `yaml:"requirements,omitempty"`
    Constraints      []string          `yaml:"constraints,omitempty"`
    Architecture     string            `yaml:"architecture,omitempty"`
    ExecutionPlan    string            `yaml:"execution_plan,omitempty"`
    Assumptions      []string          `yaml:"assumptions"`
    Uncertainty      string            `yaml:"uncertainty"`
    SourceMarkers    []SourceMarker    `yaml:"source_markers"`
    PredicateResults []PredicateResult `yaml:"predicate_results"`
    RedFlags         []RedFlag         `yaml:"red_flags"`
    AuditConclusion  string            `yaml:"audit_conclusion"`
    RawText          string            `yaml:"raw_text,omitempty"`
}

// SourceMarker 來源標記（對應 MC-UES 公理四）
type SourceMarker struct {
    Claim      string `yaml:"claim"`
    SourceType string `yaml:"source_type"` // user_input / document_ref / inference / uncertain
    Reference  string `yaml:"reference,omitempty"`
    Confidence string `yaml:"confidence"` // high / medium / low
}

// PredicateResult 判定式結果（對應 MC-UES Volume III）
type PredicateResult struct {
    PredicateName string     `yaml:"predicate_name"`
    Input         string     `yaml:"input"`
    Result        bool       `yaml:"result"`
    Evidence      []string   `yaml:"evidence,omitempty"`
    FailureReason string     `yaml:"failure_reason,omitempty"`
}

// Decision 架構決策（對應 ADR）
type Decision struct {
    ID                  string             `yaml:"id"`
    Title               string             `yaml:"title"`
    Status              DecisionStatus     `yaml:"status"`
    Version             string             `yaml:"version"`
    Constraints         []string           `yaml:"constraints"`
```

```
    ImmutableConditions  []string            `yaml:"immutable_conditions"`
    ExcludedAlternatives []ExcludedAlternative `yaml:"excluded_alternatives"`
    Tags                 []string            `yaml:"tags"`
}

// DecisionStatus 決策狀態
type DecisionStatus string

const (
    DecisionConfirmed  DecisionStatus = "confirmed"
    DecisionPending    DecisionStatus = "pending"
    DecisionSuperseded DecisionStatus = "superseded"
    DecisionDeprecated DecisionStatus = "deprecated"
)

// ExcludedAlternative 排除的替代方案
type ExcludedAlternative struct {
    Option string `yaml:"option"`
    Reason string `yaml:"reason"`
}

// TermDefinition 術語定義
type TermDefinition struct {
    Definition         string   `yaml:"definition"`
    ForbiddenSynonyms  []string `yaml:"forbidden_synonyms"`
    RequiredEvidence   []string `yaml:"required_evidence"`
    EnglishAlias       string   `yaml:"english_alias"`
}

// TermsConfig 術語表設定
type TermsConfig struct {
    Version               string                    `yaml:"version"`
    Updated               string                    `yaml:"updated"`
    Terms                 map[string]TermDefinition `yaml:"terms"`
    GlobalForbiddenPhrases []ForbiddenPhrase        `yaml:"global_forbidden_phrases"`
}

// ForbiddenPhrase 禁用詞條目
type ForbiddenPhrase struct {
    Phrase    string   `yaml:"phrase"`
    Severity  Severity `yaml:"severity"`
    MCUESRef  string   `yaml:"mc_ues_ref"`
    Reason    string   `yaml:"reason"`
}

// GatesConfig Gate 設定
```

```go
type GatesConfig struct {
    Version string      `yaml:"version"`
    Gates   GatesDetail `yaml:"gates"`
    Output  OutputConfig `yaml:"output"`
}

// GatesDetail 各 Gate 詳細設定
type GatesDetail struct {
    Gate1Structure   Gate1Config `yaml:"gate1_structure"`
    Gate2Terminology Gate2Config `yaml:"gate2_terminology"`
    Gate3RedFlags    Gate3Config `yaml:"gate3_redflags"`
    Gate4Decisions   Gate4Config `yaml:"gate4_decisions"`
    Gate5Checkpoint  Gate5Config `yaml:"gate5_checkpoint"`
}

// Gate1Config Gate 1 設定
type Gate1Config struct {
    Enabled        bool            `yaml:"enabled"`
    RequiredFields []RequiredField `yaml:"required_fields"`
}

// RequiredField 必要欄位定義
type RequiredField struct {
    Field       string `yaml:"field"`
    Description string `yaml:"description"`
    MCUESRef    string `yaml:"mc_ues_ref"`
    AllowEmpty  bool   `yaml:"allow_empty"`
}

// Gate2Config Gate 2 設定
type Gate2Config struct {
    Enabled               bool   `yaml:"enabled"`
    TermsFile             string `yaml:"terms_file"`
    FailOnForbiddenSynonym bool  `yaml:"fail_on_forbidden_synonym"`
    FailOnUndefinedTerm   bool   `yaml:"fail_on_undefined_term"`
}

// Gate3Config Gate 3 設定
type Gate3Config struct {
    Enabled           bool       `yaml:"enabled"`
    FailOnSeverity    []Severity `yaml:"fail_on_severity"`
    WarnOnSeverity    []Severity `yaml:"warn_on_severity"`
    MaxMediumWarnings int        `yaml:"max_medium_warnings"`
}

// Gate4Config Gate 4 設定
```

```go
type Gate4Config struct {
    Enabled        bool   `yaml:"enabled"`
    DecisionsDir   string `yaml:"decisions_dir"`
    FailOnConflict bool   `yaml:"fail_on_conflict"`
    WarnOnPending  bool   `yaml:"warn_on_pending"`
}

// Gate5Config Gate 5 設定
type Gate5Config struct {
    Enabled     bool              `yaml:"enabled"`
    Triggers    []CheckpointTrigger `yaml:"triggers"`
    AutoApprove []AutoApproveRule  `yaml:"auto_approve"`
}

// CheckpointTrigger 人工核查點觸發條件
type CheckpointTrigger struct {
    Condition   string `yaml:"condition"`
    Description string `yaml:"description"`
    Action      string `yaml:"action"`
}

// AutoApproveRule 自動通過規則
type AutoApproveRule struct {
    Condition   string `yaml:"condition"`
    Description string `yaml:"description"`
    Action      string `yaml:"action"`
}

// OutputConfig 輸出設定
type OutputConfig struct {
    ReportDir        string `yaml:"report_dir"`
    SnapshotDir      string `yaml:"snapshot_dir"`
    Format           string `yaml:"format"`
    IncludeTimestamp bool   `yaml:"include_timestamp"`
    IncludeVersion   bool   `yaml:"include_version"`
}

// StateSnapshot 狀態快照
type StateSnapshot struct {
    ID                  string     `yaml:"id"`
    Timestamp           time.Time  `yaml:"timestamp"`
    ValidArchitecture   string     `yaml:"valid_architecture"`
    ConfirmedDecisions  []string   `yaml:"confirmed_decisions"`
    ActiveConstraints   []string   `yaml:"active_constraints"`
    OpenProblems        []string   `yaml:"open_problems"`
    ExplicitlyExcluded  []string   `yaml:"explicitly_excluded"`
```

```
    NextCheckpoint       string     `yaml:"next_checkpoint"`
    DerivedFrom          string     `yaml:"derived_from,omitempty"`
}
```

4.3 internal/loader.go

```go
package internal

import (
    "fmt"
    "os"
    "path/filepath"

    "gopkg.in/yaml.v3"
    "github.com/yourorg/mc-ues-ci/internal/types"
)

// LoadTermsConfig 載入術語表設定
func LoadTermsConfig(path string) (*types.TermsConfig, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("無法讀取術語表 %s: %w", path, err)
    }

    var config types.TermsConfig
    if err := yaml.Unmarshal(data, &config); err != nil {
        return nil, fmt.Errorf("術語表格式錯誤 %s: %w", path, err)
    }

    return &config, nil
}

// LoadGatesConfig 載入 Gate 設定
func LoadGatesConfig(path string) (*types.GatesConfig, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("無法讀取 Gate 設定 %s: %w", path, err)
    }

    var config types.GatesConfig
    if err := yaml.Unmarshal(data, &config); err != nil {
        return nil, fmt.Errorf("Gate 設定格式錯誤 %s: %w", path, err)
    }

    return &config, nil
}

// LoadAIOutput 載入 AI 輸出檔案
func LoadAIOutput(path string) (*types.AIOutput, error) {
    data, err := os.ReadFile(path)
    if err != nil {
```

```
        return nil, fmt.Errorf("無法讀取 AI 輸出 %s: %w", path, err)
    }

    var output types.AIOutput
    if err := yaml.Unmarshal(data, &output); err != nil {
        return nil, fmt.Errorf("AI 輸出格式錯誤 %s: %w", path, err)
    }

    return &output, nil
}

// LoadAllDecisions 載入所有架構決策
func LoadAllDecisions(dir string) ([]*types.Decision, error) {
    var decisions []*types.Decision

    entries, err := os.ReadDir(dir)
    if err != nil {
        return nil, fmt.Errorf("無法讀取決策目錄 %s: %w", dir, err)
    }

    for _, entry := range entries {
        if entry.IsDir() || filepath.Ext(entry.Name()) != ".yaml" {
            continue
        }

        path := filepath.Join(dir, entry.Name())
        data, err := os.ReadFile(path)
        if err != nil {
            return nil, fmt.Errorf("無法讀取決策檔案 %s: %w", path, err)
        }

        var decision types.Decision
        if err := yaml.Unmarshal(data, &decision); err != nil {
            return nil, fmt.Errorf("決策檔案格式錯誤 %s: %w", path, err)
        }

        decisions = append(decisions, &decision)
    }

    return decisions, nil
}

// LoadLatestSnapshot 載入最新狀態快照
func LoadLatestSnapshot(dir string) (*types.StateSnapshot, error) {
    entries, err := os.ReadDir(dir)
    if err != nil {
```

```
        return nil, fmt.Errorf("無法讀取快照目錄 %s: %w", dir, err)
    }

    var latestFile string
    for _, entry := range entries {
        if !entry.IsDir() && filepath.Ext(entry.Name()) == ".yaml" {
            if entry.Name() > latestFile {
                latestFile = entry.Name()
            }
        }
    }

    if latestFile == "" {
        return nil, nil // 無快照，首次執行
    }

    path := filepath.Join(dir, latestFile)
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("無法讀取快照 %s: %w", path, err)
    }

    var snapshot types.StateSnapshot
    if err := yaml.Unmarshal(data, &snapshot); err != nil {
        return nil, fmt.Errorf("快照格式錯誤 %s: %w", path, err)
    }

    return &snapshot, nil
}
```

5. Gate 實作

5.1 internal/gate1_structure.go

```go
package internal

import (
    "fmt"
    "reflect"
    "time"

    "github.com/yourorg/mc-ues-ci/internal/types"
)

// Gate1Structure 執行結構完整性檢查
// 對應 MC-UES 14.2：標準輸出必要段落
func Gate1Structure(output *types.AIOutput, config types.Gate1Config) types.GateResult {
    start := time.Now()
    result := types.GateResult{
        GateID:    "gate1",
        GateName:  "結構完整性檢查",
        Timestamp: start,
        MCUESRefs: []string{"14.2", "14.4", "14.5", "公理四"},
    }

    if !config.Enabled {
        result.Status = types.StatusUnverified
        result.Warnings = append(result.Warnings, types.Warning{
            ID:          "G1-W000",
            Description: "Gate 1 已停用",
        })
        result.Duration = time.Since(start).Milliseconds()
        return result
    }

    var issues []types.Issue
    issueCounter := 0

    for _, field := range config.RequiredFields {
        issueCounter++
        if !fieldExists(output, field.Field) {
            issues = append(issues, types.Issue{
                ID:          fmt.Sprintf("G1-E%03d", issueCounter),
                Type:        "缺少必要欄位",
                Severity:    types.SeverityHigh,
                Description: fmt.Sprintf("缺少必要欄位：%s（%s）", field.Field, field.Description),
                MCUESRef:    field.MCUESRef,
```

```
                RedFlag:     string(types.RedFlagVerificationFailure),
            })
            continue
        }

        if !field.AllowEmpty && fieldIsEmpty(output, field.Field) {
            issues = append(issues, types.Issue{
                ID:          fmt.Sprintf("G1-E%03d", issueCounter),
                Type:        "必要欄位為空",
                Severity:    types.SeverityHigh,
                Description: fmt.Sprintf("必要欄位不得為空：%s（%s）", field.Field, field.Description),
                MCUESRef:    field.MCUESRef,
                RedFlag:     string(types.RedFlagVerificationFailure),
            })
        }
    }

    // 特別檢查：assumptions 不得為空（MC-UES 14.4）
    if len(output.Assumptions) == 0 {
        issues = append(issues, types.Issue{
            ID:          "G1-E100",
            Type:        "假設清單為空",
            Severity:    types.SeverityHigh,
            Description: "假設清單不得為空。若真無假設，必須明示「本輸出不含任何假設」",
            MCUESRef:    "14.4",
            RedFlag:     string(types.RedFlagHiddenAssumption),
        })
    }

    // 特別檢查：uncertainty 不得為空（MC-UES 14.5）
    if output.Uncertainty == "" {
        issues = append(issues, types.Issue{
            ID:          "G1-E101",
            Type:        "不確定性聲明缺失",
            Severity:    types.SeverityHigh,
            Description: "不確定性聲明不得缺失。若完全確定，必須明示「本輸出無不確定項目」並附上依據",
            MCUESRef:    "14.5",
            RedFlag:     string(types.RedFlagHiddenAssumption),
        })
    }

    // 特別檢查：source_markers 每個結論都需要來源（MC-UES 公理四）
    if len(output.SourceMarkers) == 0 {
        issues = append(issues, types.Issue{
            ID:          "G1-E102",
            Type:        "來源標記缺失",
```

```
            Severity:    types.SeverityHigh,
            Description: "所有結論必須附有來源標記。來源類型：user_input / document_ref / inference / uncertain",
            MCUESRef:    "公理四",
            RedFlag:     string(types.RedFlagEvidenceMissing),
        })
    } else {
        // 檢查每個來源標記是否有有效的來源類型
        validSourceTypes := map[string]bool{
            "user_input":    true,
            "document_ref":  true,
            "inference":     true,
            "uncertain":     true,
        }
        for i, sm := range output.SourceMarkers {
            if !validSourceTypes[sm.SourceType] {
                issues = append(issues, types.Issue{
                    ID:          fmt.Sprintf("G1-E%03d", 110+i),
                    Type:        "無效來源類型",
                    Severity:    types.SeverityMedium,
                    Description: fmt.Sprintf("來源標記 #%d 使用了無效的來源類型：%s", i+1, sm.SourceType),
                    MCUESRef:    "公理四",
                })
            }
        }
    }

    result.Issues = issues
    result.Duration = time.Since(start).Milliseconds()

    if len(issues) == 0 {
        result.Status = types.StatusPass
    } else {
        result.Status = types.StatusFail
    }

    return result
}

// fieldExists 檢查 AIOutput 中指定欄位是否存在
func fieldExists(output *types.AIOutput, fieldName string) bool {
    v := reflect.ValueOf(output).Elem()
    t := v.Type()

    for i := 0; i < t.NumField(); i++ {
        tag := t.Field(i).Tag.Get("yaml")
```

```
        if tag == fieldName || tag == fieldName+",omitempty" {
            return true
        }
    }
    return false
}

// fieldIsEmpty 檢查 AIOutput 中指定欄位是否為空
func fieldIsEmpty(output *types.AIOutput, fieldName string) bool {
    v := reflect.ValueOf(output).Elem()
    t := v.Type()

    for i := 0; i < t.NumField(); i++ {
        tag := t.Field(i).Tag.Get("yaml")
        if tag == fieldName || tag == fieldName+",omitempty" {
            field := v.Field(i)
            switch field.Kind() {
            case reflect.String:
                return field.String() == ""
            case reflect.Slice:
                return field.Len() == 0
            case reflect.Map:
                return field.Len() == 0
            }
        }
    }
    return false
}
```

5.2 internal/gate2_terminology.go

```go
package internal

import (
    "fmt"
    "strings"
    "time"

    "github.com/yourorg/mc-ues-ci/internal/types"
)

// Gate2Terminology 執行術語邊界檢查
// 對應 MC-UES 2.5：單字多義控制、0.5：語義偏移禁止
func Gate2Terminology(output *types.AIOutput, terms *types.TermsConfig, config types.Gate2Config)
        types.GateResult {
    start := time.Now()
    result := types.GateResult{
        GateID:    "gate2",
        GateName:  "術語邊界檢查",
        Timestamp: start,
        MCUESRefs: []string{"0.5", "2.5", "16"},
    }

    if !config.Enabled {
        result.Status = types.StatusUnverified
        result.Duration = time.Since(start).Milliseconds()
        return result
    }

    var issues []types.Issue
    var warnings []types.Warning
    issueCounter := 0
    warnCounter := 0

    // 取得所有文字內容進行掃描
    textToScan := collectAllText(output)

    // 檢查全域禁用詞（MC-UES 12.3）
    for _, forbidden := range terms.GlobalForbiddenPhrases {
        if containsPhrase(textToScan, forbidden.Phrase) {
            locations := findAllLocations(textToScan, forbidden.Phrase)
            for _, loc := range locations {
                issueCounter++
                issue := types.Issue{
                    ID:          fmt.Sprintf("G2-E%03d", issueCounter),
                    Type:        "禁用詞使用",
```

```
                Description: fmt.Sprintf("發現禁用詞「%s」：%s", forbidden.Phrase, forbidden.Reason),
                Location:    loc,
                MCUESRef:    forbidden.MCUESRef,
                RedFlag:     string(types.RedFlagNonDeterministicReason),
            }

            if forbidden.Severity == types.SeverityHigh {
                issue.Severity = types.SeverityHigh
                issues = append(issues, issue)
            } else {
                issue.Severity = types.SeverityMedium
                issues = append(issues, issue)
            }
        }
    }
}

// 檢查各術語的禁用同義詞
for termName, termDef := range terms.Terms {
    for _, synonym := range termDef.ForbiddenSynonyms {
        if containsPhrase(textToScan, synonym) {
            locations := findAllLocations(textToScan, synonym)
            for _, loc := range locations {
                issueCounter++
                issues = append(issues, types.Issue{
                    ID:          fmt.Sprintf("G2-E%03d", issueCounter),
                    Type:        "術語同義詞使用",
                    Severity:    types.SeverityMedium,
                    Description: fmt.Sprintf("使用了術語「%s」的禁用同義詞「%s」，請改用正式術語",
 termName, synonym),
                    Location:    loc,
                    MCUESRef:    "16",
                    RedFlag:     string(types.RedFlagSemanticDrift),
                })
            }
        }
    }
}

// 檢查未定義術語（警告模式）
if config.FailOnUndefinedTerm {
    usedTerms := extractTerms(textToScan)
    for _, term := range usedTerms {
        if _, defined := terms.Terms[term]; !defined {
            warnCounter++
            warnings = append(warnings, types.Warning{
                ID:          fmt.Sprintf("G2-W%03d", warnCounter),
```

```
                    Description: fmt.Sprintf("使用了未在術語表中定義的術語：「%s」", term),
                    MCUESRef:    "16",
                })
            }
        }
    }

    result.Issues = issues
    result.Warnings = warnings
    result.Duration = time.Since(start).Milliseconds()

    // 判定 Gate 狀態
    highIssues := countBySeverity(issues, types.SeverityHigh)
    if highIssues > 0 && config.FailOnForbiddenSynonym {
        result.Status = types.StatusFail
    } else if len(issues) > 0 {
        result.Status = types.StatusConditionalFail
    } else if len(warnings) > 0 {
        result.Status = types.StatusConditionalPass
    } else {
        result.Status = types.StatusPass
    }

    return result
}

// collectAllText 收集 AIOutput 中所有文字內容
func collectAllText(output *types.AIOutput) string {
    var parts []string

    parts = append(parts, output.IntentSummary)
    parts = append(parts, output.Architecture)
    parts = append(parts, output.ExecutionPlan)
    parts = append(parts, output.Uncertainty)
    parts = append(parts, output.AuditConclusion)
    parts = append(parts, output.RawText)

    for _, req := range output.Requirements {
        parts = append(parts, req)
    }
    for _, con := range output.Constraints {
        parts = append(parts, con)
    }
    for _, asm := range output.Assumptions {
        parts = append(parts, asm)
    }
```

```go
    for _, sm := range output.SourceMarkers {
        parts = append(parts, sm.Claim)
        parts = append(parts, sm.Reference)
    }
    for _, pr := range output.PredicateResults {
        parts = append(parts, pr.Input)
        parts = append(parts, pr.FailureReason)
    }
    for _, rf := range output.RedFlags {
        parts = append(parts, rf.Description)
    }

    return strings.Join(parts, "\n")
}

// containsPhrase 檢查文字中是否包含指定詞組
func containsPhrase(text, phrase string) bool {
    return strings.Contains(text, phrase)
}

// findAllLocations 找出所有出現位置的摘要
func findAllLocations(text, phrase string) []string {
    var locations []string
    lines := strings.Split(text, "\n")

    for i, line := range lines {
        if strings.Contains(line, phrase) {
            // 截取上下文（最多 60 字元）
            context := line
            if len(context) > 60 {
                idx := strings.Index(context, phrase)
                start := idx - 20
                if start < 0 {
                    start = 0
                }
                end := idx + len(phrase) + 20
                if end > len(context) {
                    end = len(context)
                }
                context = "..." + context[start:end] + "..."
            }
            locations = append(locations, fmt.Sprintf("第 %d 行：%s", i+1, context))
        }
    }

    return locations
```

```go
}

// extractTerms 從文字中提取可能的術語（簡化版）
func extractTerms(text string) []string {
    // 這裡實作簡化版的術語提取
    // 實際專案可以使用更複雜的 NLP 方法
    var terms []string
    // 目前返回空集合，需要根據專案實際需求實作
    return terms
}

// countBySeverity 計算指定嚴重程度的問題數量
func countBySeverity(issues []types.Issue, severity types.Severity) int {
    count := 0
    for _, issue := range issues {
        if issue.Severity == severity {
            count++
        }
    }
    return count
}
```

5.3 internal/gate3_redflags.go

```go
package internal

import (
    "fmt"
    "strings"
    "time"

    "github.com/yourorg/mc-ues-ci/internal/types"
)

// Gate3RedFlags 執行紅旗偵測
// 對應 MC-UES Volume VII：錯誤分類系統
func Gate3RedFlags(output *types.AIOutput, config types.Gate3Config) types.GateResult {
    start := time.Now()
    result := types.GateResult{
        GateID:    "gate3",
        GateName:  "紅旗偵測",
        Timestamp: start,
        MCUESRefs: []string{"8.1", "8.2", "8.3", "8.4", "8.5"},
    }

    if !config.Enabled {
        result.Status = types.StatusUnverified
        result.Duration = time.Since(start).Milliseconds()
        return result
    }

    var issues []types.Issue
    var warnings []types.Warning
    issueCounter := 0

    textToScan := collectAllText(output)

    // 1. 非決定性推理偵測（MC-UES 12.3）
    ndrPatterns := []struct {
        pattern string
        context string
    }{
        {"大概", "使用非決定性詞彙作為判定依據"},
        {"應該可以", "使用非決定性詞彙作為判定依據"},
        {"看起來合理", "使用印象替代形式證明"},
        {"probably", "Non-deterministic reasoning"},
        {"should work", "Non-deterministic reasoning"},
        {"seems reasonable", "Non-deterministic reasoning"},
        {"差不多", "使用模糊詞彙"},
```

```
        {"感覺上", "使用主觀感受替代形式判定"},
        {"直覺告訴", "使用直覺替代判定式"},
        {"經驗上", "使用經驗替代可驗證證據"},
    }

    for _, pattern := range ndrPatterns {
        if containsPhrase(textToScan, pattern.pattern) {
            issueCounter++
            issues = append(issues, types.Issue{
                ID:          fmt.Sprintf("G3-E%03d", issueCounter),
                Type:        "非決定性推理",
                Severity:    types.SeverityHigh,
                Description: fmt.Sprintf("發現非決定性推理：「%s」— %s", pattern.pattern,
     pattern.context),
                MCUESRef:    "12.3",
                RedFlag:     string(types.RedFlagNonDeterministicReason),
            })
        }
    }

    // 2. 隱含假設偵測（MC-UES 14.4）
    // 檢查結論是否缺乏前提
    for i, pr := range output.PredicateResults {
        if pr.Result && len(pr.Evidence) == 0 {
            issueCounter++
            issues = append(issues, types.Issue{
                ID:          fmt.Sprintf("G3-E%03d", issueCounter),
                Type:        "判定式無證據",
                Severity:    types.SeverityHigh,
                Description: fmt.Sprintf("判定式 #%d「%s」結果為真但無證據支持", i+1, pr.PredicateName),
                MCUESRef:    "公理二",
                RedFlag:     string(types.RedFlagHiddenAssumption),
            })
        }
    }

    // 3. 虛構依賴偵測（MC-UES 8.2）
    // 偵測引用了不存在的文件、函式或模組
    hallucinationPatterns := []string{
        "如前所述",          // 可能引用了不存在的前文
        "眾所周知",          // 以印象替代證明
        "業界標準做法",      // 未引用具體來源
        "根據最佳實踐",      // 未引用具體來源
        "as mentioned above", // May reference non-existent content
        "it is well known",   // 以印象替代證明
    }
```

```
for _, pattern := range hallucinationPatterns {
    if containsPhrase(textToScan, pattern) {
        issueCounter++
        issues = append(issues, types.Issue{
            ID:          fmt.Sprintf("G3-E%03d", issueCounter),
            Type:        "潛在虛構依賴",
            Severity:    types.SeverityMedium,
            Description: fmt.Sprintf("發現可能的虛構依賴：「%s」— 請提供具體可追溯來源", pattern),
            MCUESRef:    "12.2",
            RedFlag:     string(types.RedFlagHallucinatedDependency),
        })
    }
}

// 4. 語義偏移偵測（MC-UES 0.5）
// 檢查輸出中的術語使用是否一致
termUsage := detectTermInconsistency(textToScan)
for _, inconsistency := range termUsage {
    issueCounter++
    issues = append(issues, types.Issue{
        ID:          fmt.Sprintf("G3-E%03d", issueCounter),
        Type:        "術語使用不一致",
        Severity:    types.SeverityMedium,
        Description: inconsistency,
        MCUESRef:    "0.5",
        RedFlag:     string(types.RedFlagSemanticDrift),
    })
}

// 5. 驗證失敗隱藏偵測（MC-UES 8.5, 12.4）
hidingPatterns := []string{
    "雖然有些問題",     // 淡化失敗
    "但整體而言",       // 掩蓋問題
    "小瑕疵",           // 降低問題嚴重性
    "不影響大局",       // 隱藏失敗
    "minor issues",    // 淡化失敗
    "overall it works", // 掩蓋問題
}

for _, pattern := range hidingPatterns {
    if containsPhrase(textToScan, pattern) {
        issueCounter++
        issues = append(issues, types.Issue{
            ID:          fmt.Sprintf("G3-E%03d", issueCounter),
            Type:        "潛在失敗隱藏",
            Severity:    types.SeverityHigh,
```

```
            Description: fmt.Sprintf("發現可能的失敗隱藏行為：「%s」— 所有失敗項必須明示（MC-UES 12.4）", pattern),
            MCUESRef:    "12.4",
            RedFlag:     string(types.RedFlagVerificationFailure),
        })
    }
}

// 6. 未驗證偽裝已驗證偵測（MC-UES 12.5）
for _, pr := range output.PredicateResults {
    if pr.Result && pr.FailureReason == "" && len(pr.Evidence) == 0 {
        issueCounter++
        issues = append(issues, types.Issue{
            ID:          fmt.Sprintf("G3-E%03d", issueCounter),
            Type:        "未驗證偽裝已驗證",
            Severity:    types.SeverityHigh,
            Description: fmt.Sprintf("判定式「%s」標記為通過但無任何證據或推理依據", pr.PredicateName),
            MCUESRef:    "12.5",
            RedFlag:     string(types.RedFlagPredicateViolation),
        })
    }
}

// 將 output 中已有的紅旗也納入計算
for _, rf := range output.RedFlags {
    issueCounter++
    issues = append(issues, types.Issue{
        ID:          fmt.Sprintf("G3-E%03d", issueCounter),
        Type:        "輸出自報紅旗",
        Severity:    rf.Severity,
        Description: fmt.Sprintf("[自報] %s：%s", rf.Type, rf.Description),
        MCUESRef:    rf.MCUESRef,
        RedFlag:     string(rf.Type),
    })
}

result.Issues = issues
result.Warnings = warnings
result.Duration = time.Since(start).Milliseconds()

// 判定 Gate 狀態
highCount := countBySeverity(issues, types.SeverityHigh)
mediumCount := countBySeverity(issues, types.SeverityMedium)

shouldFail := false
for _, sev := range config.FailOnSeverity {
```

```
        if sev == types.SeverityHigh && highCount > 0 {
            shouldFail = true
        }
        if sev == types.SeverityMedium && mediumCount > 0 {
            shouldFail = true
        }
    }

    if mediumCount > config.MaxMediumWarnings {
        shouldFail = true
    }

    if shouldFail {
        result.Status = types.StatusFail
    } else if mediumCount > 0 {
        result.Status = types.StatusConditionalPass
    } else if highCount == 0 && mediumCount == 0 {
        result.Status = types.StatusPass
    } else {
        result.Status = types.StatusConditionalFail
    }

    return result
}

// detectTermInconsistency 偵測術語使用不一致性
func detectTermInconsistency(text string) []string {
    var inconsistencies []string

    // 術語對應的常見混用情況
    termPairs := []struct {
        term1   string
        term2   string
        message string
    }{
        {"需求", "requirement", "中英文術語混用，請統一使用中文術語或在術語表中明示別名"},
        {"約束", "constraint", "中英文術語混用，請統一使用中文術語或在術語表中明示別名"},
        {"驗證", "verification", "中英文術語混用（注意：validation 和 verification 語義不同）"},
    }

    for _, pair := range termPairs {
        hasTerm1 := containsPhrase(text, pair.term1)
        hasTerm2 := containsPhrase(text, strings.ToLower(pair.term2))
        if hasTerm1 && hasTerm2 {
            inconsistencies = append(inconsistencies,
                fmt.Sprintf("同一輸出中混用「%s」和「%s」：%s", pair.term1, pair.term2, pair.message))
```

```
        }
    }

    return inconsistencies
}
```

5.4 internal/gate4_decisions.go

```go
package internal

import (
    "fmt"
    "strings"
    "time"

    "github.com/yourorg/mc-ues-ci/internal/types"
)

// Gate4Decisions 執行決策日誌一致性檢查
// 對應 MC-UES 公理六：以建構達成符合
func Gate4Decisions(output *types.AIOutput, decisions []*types.Decision, config types.Gate4Config)
        types.GateResult {
    start := time.Now()
    result := types.GateResult{
        GateID:    "gate4",
        GateName:  "決策日誌一致性檢查",
        Timestamp: start,
        MCUESRefs: []string{"公理六", "13.1", "13.2", "13.3"},
    }

    if !config.Enabled {
        result.Status = types.StatusUnverified
        result.Duration = time.Since(start).Milliseconds()
        return result
    }

    var issues []types.Issue
    var warnings []types.Warning
    issueCounter := 0
    warnCounter := 0

    textToScan := collectAllText(output)

    for _, decision := range decisions {
        switch decision.Status {
        case types.DecisionConfirmed:
            // 檢查輸出是否與已確認決策衝突
            conflicts := checkDecisionConflicts(textToScan, output, decision)
            for _, conflict := range conflicts {
                issueCounter++
                issues = append(issues, types.Issue{
                    ID:          fmt.Sprintf("G4-E%03d", issueCounter),
                    Type:        "決策衝突",
```

```
            Severity:    types.SeverityHigh,
            Description: fmt.Sprintf("輸出與已確認決策 %s（%s）衝突：%s", decision.ID,
 decision.Title, conflict),
            MCUESRef:    "13.2",
            RedFlag:     string(types.RedFlagArchitectureInconsist),
        })
    }

    // 檢查是否違反不可變條件
    for _, immutable := range decision.ImmutableConditions {
        if violatesImmutableCondition(textToScan, immutable) {
            issueCounter++
            issues = append(issues, types.Issue{
                ID:          fmt.Sprintf("G4-E%03d", issueCounter),
                Type:        "不可變條件違反",
                Severity:    types.SeverityHigh,
                Description: fmt.Sprintf("輸出違反決策 %s 的不可變條件：「%s」", decision.ID,
 immutable),
                MCUESRef:    "13.3",
                RedFlag:     string(types.RedFlagConstraintConflict),
            })
        }
    }

    // 檢查是否提議使用已排除的替代方案
    for _, excluded := range decision.ExcludedAlternatives {
        if mentionsExcludedOption(textToScan, excluded.Option) {
            issueCounter++
            issues = append(issues, types.Issue{
                ID:          fmt.Sprintf("G4-E%03d", issueCounter),
                Type:        "提議排除方案",
                Severity:    types.SeverityMedium,
                Description: fmt.Sprintf("輸出提議使用已在決策 %s 中排除的方案「%s」（排除理由：
%s）", decision.ID, excluded.Option, excluded.Reason),
                MCUESRef:    "13.3",
                RedFlag:     string(types.RedFlagArchitectureInconsist),
            })
        }
    }

case types.DecisionPending:
    if config.WarnOnPending {
        warnCounter++
        warnings = append(warnings, types.Warning{
            ID:          fmt.Sprintf("G4-W%03d", warnCounter),
            Description: fmt.Sprintf("決策 %s（%s）狀態為 pending，相關輸出可能在決策確認後需要修
訂", decision.ID, decision.Title),
            MCUESRef:    "13.1",
        })
```

```
            }

        case types.DecisionSuperseded, types.DecisionDeprecated:
            // 檢查是否仍在引用已廢棄的決策
            if mentionsDecisionID(textToScan, decision.ID) {
                issueCounter++
                issues = append(issues, types.Issue{
                    ID:          fmt.Sprintf("G4-E%03d", issueCounter),
                    Type:        "引用廢棄決策",
                    Severity:    types.SeverityMedium,
                    Description: fmt.Sprintf("輸出引用了已廢棄的決策 %s（狀態：%s）", decision.ID,
 decision.Status),
                    MCUESRef:    "13.2",
                    RedFlag:     string(types.RedFlagRequirementMutation),
                })
            }
        }
    }

    result.Issues = issues
    result.Warnings = warnings
    result.Duration = time.Since(start).Milliseconds()

    highCount := countBySeverity(issues, types.SeverityHigh)
    if highCount > 0 && config.FailOnConflict {
        result.Status = types.StatusFail
    } else if len(issues) > 0 {
        result.Status = types.StatusConditionalFail
    } else if len(warnings) > 0 {
        result.Status = types.StatusConditionalPass
    } else {
        result.Status = types.StatusPass
    }

    return result
}

// checkDecisionConflicts 檢查輸出是否與已確認決策衝突
func checkDecisionConflicts(text string, output *types.AIOutput, decision *types.Decision) []string {
    var conflicts []string

    // 檢查輸出的約束是否與決策約束衝突
    for _, outputConstraint := range output.Constraints {
        for _, decisionConstraint := range decision.Constraints {
            if isConflicting(outputConstraint, decisionConstraint) {
                conflicts = append(conflicts,
```

```
                    fmt.Sprintf("輸出約束「%s」與決策約束「%s」衝突", outputConstraint,
        decisionConstraint))
            }
        }
    }

    return conflicts
}

// isConflicting 判斷兩個約束是否衝突（簡化版）
// 實際專案需要根據領域知識實作更精確的衝突偵測
func isConflicting(constraint1, constraint2 string) bool {
    // 目前實作：檢查明顯的否定關係
    negationPairs := []struct{ a, b string }{
        {"不得使用", "必須使用"},
        {"禁止", "必須"},
        {"不允許", "要求"},
    }

    for _, pair := range negationPairs {
        if strings.Contains(constraint1, pair.a) && strings.Contains(constraint2, pair.b) {
            // 提取主詞並比較
            subject1 := extractSubject(constraint1, pair.a)
            subject2 := extractSubject(constraint2, pair.b)
            if subject1 != "" && subject2 != "" && strings.Contains(subject1, subject2) {
                return true
            }
        }
    }

    return false
}

// extractSubject 從約束句子中提取主詞（簡化版）
func extractSubject(constraint, marker string) string {
    idx := strings.Index(constraint, marker)
    if idx < 0 {
        return ""
    }
    after := strings.TrimSpace(constraint[idx+len(marker):])
    // 取第一個詞組作為主詞
    parts := strings.Fields(after)
    if len(parts) > 0 {
        return parts[0]
    }
    return ""
}
```

```go
// violatesImmutableCondition 檢查文字是否違反不可變條件
func violatesImmutableCondition(text, condition string) bool {
    // 提取條件的核心動作
    // 例如：「不得在未升版此 ADR 的情況下，不得更換資料庫引擎」
    // 偵測文字中是否出現「更換資料庫引擎」相關內容
    keywords := extractKeywords(condition)
    matchCount := 0
    for _, kw := range keywords {
        if containsPhrase(text, kw) {
            matchCount++
        }
    }
    return matchCount >= 2 // 至少兩個關鍵詞同時出現才視為違反
}

// mentionsExcludedOption 檢查是否提議使用排除的方案
func mentionsExcludedOption(text, option string) bool {
    return containsPhrase(text, option)
}

// mentionsDecisionID 檢查是否引用了特定決策 ID
func mentionsDecisionID(text, id string) bool {
    return containsPhrase(text, id)
}

// extractKeywords 從句子中提取關鍵詞
func extractKeywords(sentence string) []string {
    // 移除常見助詞和連接詞
    stopWords := []string{"的", "在", "且", "並", "或", "與", "及", "不得", "必須", "應該", "不", "情況
        下"}
    words := strings.Fields(sentence)
    var keywords []string
    for _, word := range words {
        isStop := false
        for _, stop := range stopWords {
            if word == stop {
                isStop = true
                break
            }
        }
        if !isStop && len([]rune(word)) >= 2 {
            keywords = append(keywords, word)
        }
    }
```

```
    return keywords
}
```

5.5 internal/gate5_checkpoint.go

```go
package internal

import (
    "bufio"
    "fmt"
    "os"
    "strings"
    "time"

    "github.com/yourorg/mc-ues-ci/internal/types"
)

// Gate5Checkpoint 執行人工核查點
// 對應 MC-UES 9.1：一致性定義、9.6：審計門檻
func Gate5Checkpoint(
    output *types.AIOutput,
    gateResults []types.GateResult,
    decisions []*types.Decision,
    config types.Gate5Config,
    nonInteractive bool,
) types.GateResult {
    start := time.Now()
    result := types.GateResult{
        GateID:    "gate5",
        GateName:  "人工核查點",
        Timestamp: start,
        MCUESRefs: []string{"9.1", "9.6", "15.3"},
    }

    if !config.Enabled {
        result.Status = types.StatusUnverified
        result.Duration = time.Since(start).Milliseconds()
        return result
    }

    // 評估是否觸發人工核查點
    triggeredConditions := evaluateTriggers(output, gateResults, decisions, config)

    if len(triggeredConditions) == 0 {
        // 無觸發條件，自動通過
        result.Status = types.StatusPass
        result.Warnings = append(result.Warnings, types.Warning{
            ID:          "G5-W001",
            Description: "所有自動核查通過，無需人工介入",
        })
```

```
    result.Duration = time.Since(start).Milliseconds()
    return result
}

// 有觸發條件
var issues []types.Issue
for i, condition := range triggeredConditions {
    issues = append(issues, types.Issue{
        ID:          fmt.Sprintf("G5-T%03d", i+1),
        Type:        "人工核查點觸發",
        Severity:    types.SeverityHigh,
        Description: condition,
        MCUESRef:    "9.6",
    })
}
result.Issues = issues

if nonInteractive {
    // 非互動模式：標記為等待人工確認
    result.Status = types.StatusPendingHuman
    result.Duration = time.Since(start).Milliseconds()
    return result
}

// 互動模式：請求人工確認
fmt.Println("\n" + strings.Repeat("=", 60))
fmt.Println("【MC-UES 人工核查點】")
fmt.Println(strings.Repeat("=", 60))
fmt.Println("\n以下條件觸發了人工核查點：\n")

for i, condition := range triggeredConditions {
    fmt.Printf("  %d. %s\n", i+1, condition)
}

fmt.Println("\n輸出摘要：")
fmt.Printf("  意圖：%s\n", output.IntentSummary)
fmt.Printf("  假設數量：%d\n", len(output.Assumptions))
fmt.Printf("  來源標記數量：%d\n", len(output.SourceMarkers))
fmt.Printf("  判定式結果數量：%d\n", len(output.PredicateResults))

fmt.Println("\n請確認是否批准此輸出進入下一階段：")
fmt.Println("  [y] 批准（APPROVE）")
fmt.Println("  [n] 拒絕（REJECT）")
fmt.Println("  [s] 跳過核查點（SKIP，僅用於開發測試）")
fmt.Print("\n請輸入選擇：")
```

```
    reader := bufio.NewReader(os.Stdin)
    input, _ := reader.ReadString('\n')
    input = strings.TrimSpace(strings.ToLower(input))

    switch input {
    case "y", "yes", "approve":
        result.Status = types.StatusPass
        result.Warnings = append(result.Warnings, types.Warning{
            ID:          "G5-W002",
            Description: "人工核查點已批准",
        })
    case "n", "no", "reject":
        result.Status = types.StatusFail
        result.Issues = append(result.Issues, types.Issue{
            ID:          "G5-E001",
            Type:        "人工核查拒絕",
            Severity:    types.SeverityHigh,
            Description: "人工核查點拒絕此輸出",
            MCUESRef:    "9.6",
        })
    case "s", "skip":
        result.Status = types.StatusConditionalPass
        result.Warnings = append(result.Warnings, types.Warning{
            ID:          "G5-W003",
            Description: "人工核查點已跳過（開發測試模式）",
        })
    default:
        result.Status = types.StatusFail
        result.Issues = append(result.Issues, types.Issue{
            ID:          "G5-E002",
            Type:        "無效輸入",
            Severity:    types.SeverityHigh,
            Description: fmt.Sprintf("人工核查點收到無效輸入：%s", input),
        })
    }

    result.Duration = time.Since(start).Milliseconds()
    return result
}

// evaluateTriggers 評估哪些觸發條件成立
func evaluateTriggers(
    output *types.AIOutput,
    gateResults []types.GateResult,
    decisions []*types.Decision,
    config types.Gate5Config,
```

```
) []string {
    var triggered []string

    for _, trigger := range config.Triggers {
        switch trigger.Condition {
        case "new_constraint_affects_confirmed_decision":
            if newConstraintAffectsConfirmedDecision(output, decisions) {
                triggered = append(triggered,
                    fmt.Sprintf("%s：%s", trigger.Description, trigger.Condition))
            }

        case "new_exclusion_decision":
            if hasNewExclusionDecision(output) {
                triggered = append(triggered,
                    fmt.Sprintf("%s：輸出中包含新的排除決策建議", trigger.Description))
            }

        case "architecture_direction_change":
            if hasArchitectureDirectionChange(output, decisions) {
                triggered = append(triggered,
                    fmt.Sprintf("%s：偵測到架構方向可能變更", trigger.Description))
            }

        case "high_red_flag_after_gate3":
            if hasHighRedFlagInGate3(gateResults) {
                triggered = append(triggered,
                    fmt.Sprintf("%s：Gate 3 偵測到 HIGH 嚴重度紅旗", trigger.Description))
            }
        }
    }

    return triggered
}

func newConstraintAffectsConfirmedDecision(output *types.AIOutput, decisions []*types.Decision) bool {
    for _, constraint := range output.Constraints {
        for _, decision := range decisions {
            if decision.Status == types.DecisionConfirmed {
                for _, dc := range decision.Constraints {
                    if isRelatedConstraint(constraint, dc) {
                        return true
                    }
                }
            }
        }
    }
```

```
    return false
}

func isRelatedConstraint(newConstraint, existingConstraint string) bool {
    newKW := extractKeywords(newConstraint)
    existingKW := extractKeywords(existingConstraint)
    matchCount := 0
    for _, nkw := range newKW {
        for _, ekw := range existingKW {
            if nkw == ekw {
                matchCount++
            }
        }
    }
    return matchCount >= 2
}

func hasNewExclusionDecision(output *types.AIOutput) bool {
    exclusionKeywords := []string{"排除", "不採用", "放棄", "不使用", "棄用", "exclude", "reject"}
    text := collectAllText(output)
    for _, kw := range exclusionKeywords {
        if containsPhrase(text, kw) {
            return true
        }
    }
    return false
}

func hasArchitectureDirectionChange(output *types.AIOutput, decisions []*types.Decision) bool {
    changeKeywords := []string{"改變架構", "重新設計", "重構", "架構調整", "架構變更", "refactor",
        "redesign"}
    text := collectAllText(output)
    for _, kw := range changeKeywords {
        if containsPhrase(text, kw) {
            return true
        }
    }
    return false
}

func hasHighRedFlagInGate3(gateResults []types.GateResult) bool {
    for _, gr := range gateResults {
        if gr.GateID == "gate3" {
            for _, issue := range gr.Issues {
                if issue.Severity == types.SeverityHigh {
                    return true
                }
```

```
            }
        }
    }
    return false
}
```

6. 報告系統

6.1 `internal/report.go`

```go
package internal

import (
    "crypto/sha256"
    "fmt"
    "os"
    "path/filepath"
    "time"

    "gopkg.in/yaml.v3"
    "github.com/yourorg/mc-ues-ci/internal/types"
)

// GenerateAuditReport 產生完整審計報告
func GenerateAuditReport(
    inputFile string,
    output *types.AIOutput,
    gateResults []types.GateResult,
    config *types.GatesConfig,
) *types.AuditReport {
    report := &types.AuditReport{
        ReportID:    generateReportID(),
        SpecVersion: "MC-UES-1.0",
        Timestamp:   time.Now(),
        InputFile:   inputFile,
        InputHash:   computeFileHash(inputFile),
        GateResults: gateResults,
    }

    // 收集所有紅旗
    var allRedFlags []types.RedFlag
    for _, gr := range gateResults {
        for _, issue := range gr.Issues {
            if issue.RedFlag != "" {
                allRedFlags = append(allRedFlags, types.RedFlag{
                    Type:        types.RedFlagType(issue.RedFlag),
                    Severity:    issue.Severity,
                    Description: issue.Description,
                    Location:    issue.Location,
                    MCUESRef:    issue.MCUESRef,
                })
            }
        }
```

```
    }
    report.RedFlags = allRedFlags

    // 判定總體狀態（對應 MC-UES 17：FullyConformant）
    report.OverallStatus = computeOverallStatus(gateResults)
    report.TraceComplete = isTraceComplete(output)
    report.EvidenceSufficient = isEvidenceSufficient(output)

    // 產生審計結論
    report.AuditConclusion = generateAuditConclusion(report)

    return report
}

// computeOverallStatus 計算總體狀態
// 對應 MC-UES 17：FullyConformant(S) ↔ 所有關鍵判定式通過 ∧ 無關鍵紅旗 ∧ 證據充分 ∧ 追溯完整
func computeOverallStatus(gateResults []types.GateResult) types.GateStatus {
    hasFailure := false
    hasConditional := false
    hasUnverified := false
    hasPendingHuman := false

    for _, gr := range gateResults {
        switch gr.Status {
        case types.StatusFail:
            hasFailure = true
        case types.StatusConditionalFail, types.StatusConditionalPass:
            hasConditional = true
        case types.StatusUnverified:
            hasUnverified = true
        case types.StatusPendingHuman:
            hasPendingHuman = true
        }
    }

    if hasFailure {
        return types.StatusFail // 不符合
    }
    if hasPendingHuman {
        return types.StatusPendingHuman // 等待人工
    }
    if hasUnverified {
        return types.StatusUnverified // 不可驗證
    }
    if hasConditional {
        return types.StatusConditionalPass // 條件符合
```

```
    }
    return types.StatusPass // 完全符合
}

// isTraceComplete 檢查追溯完整性
func isTraceComplete(output *types.AIOutput) bool {
    if len(output.SourceMarkers) == 0 {
        return false
    }
    for _, sm := range output.SourceMarkers {
        if sm.SourceType == "" || sm.Claim == "" {
            return false
        }
    }
    return true
}

// isEvidenceSufficient 檢查證據充分性
func isEvidenceSufficient(output *types.AIOutput) bool {
    if len(output.PredicateResults) == 0 {
        return false
    }
    for _, pr := range output.PredicateResults {
        if pr.Result && len(pr.Evidence) == 0 {
            return false
        }
    }
    return true
}

// generateAuditConclusion 產生審計結論文字
func generateAuditConclusion(report *types.AuditReport) string {
    statusMap := map[types.GateStatus]string{
        types.StatusPass:            "完全符合（Fully Conformant）",
        types.StatusFail:            "不符合（Nonconformant）",
        types.StatusConditionalPass: "條件符合（Conditionally Conformant）",
        types.StatusConditionalFail: "條件不符合（Conditionally Nonconformant）",
        types.StatusUnverified:      "不可驗證（Unverifiable）",
        types.StatusPendingHuman:    "等待人工核查（Pending Human Review）",
    }

    conclusion := fmt.Sprintf("MC-UES 審計結論：%s\n", statusMap[report.OverallStatus])
    conclusion += fmt.Sprintf("報告 ID：%s\n", report.ReportID)
    conclusion += fmt.Sprintf("時間戳記：%s\n", report.Timestamp.Format("2006-01-02T15:04:05Z07:00"))
    conclusion += fmt.Sprintf("追溯完整：%v | 證據充分：%v\n", report.TraceComplete,
        report.EvidenceSufficient)
    conclusion += fmt.Sprintf("紅旗數量：%d\n", len(report.RedFlags))
```

```
    gateSummary := ""
    for _, gr := range report.GateResults {
        gateSummary += fmt.Sprintf("  %s (%s)：%s\n", gr.GateID, gr.GateName, gr.Status)
    }
    conclusion += "Gate 摘要：\n" + gateSummary

    return conclusion
}

// SaveReport 儲存審計報告
func SaveReport(report *types.AuditReport, outputDir string) (string, error) {
    if err := os.MkdirAll(outputDir, 0755); err != nil {
        return "", fmt.Errorf("無法建立報告目錄：%w", err)
    }

    filename := fmt.Sprintf("audit-%s-%s.yaml",
        report.Timestamp.Format("20060102-150405"),
        report.ReportID[:8])
    path := filepath.Join(outputDir, filename)

    data, err := yaml.Marshal(report)
    if err != nil {
        return "", fmt.Errorf("無法序列化報告：%w", err)
    }

    if err := os.WriteFile(path, data, 0644); err != nil {
        return "", fmt.Errorf("無法寫入報告：%w", err)
    }

    return path, nil
}

// PrintReport 在終端機顯示報告
func PrintReport(report *types.AuditReport) {
    statusSymbol := map[types.GateStatus]string{
        types.StatusPass:            "✅",
        types.StatusFail:            "❌",
        types.StatusConditionalPass: "⚠️",
        types.StatusConditionalFail: "⚠️",
        types.StatusUnverified:      "❓",
        types.StatusPendingHuman:    "⏳",
    }

    fmt.Println("\n" + strings.Repeat("=", 60))
    fmt.Println("MC-UES 審計報告")
```

```
    fmt.Println(strings.Repeat("=", 60))
    fmt.Printf("報告 ID：%s\n", report.ReportID)
    fmt.Printf("時間：%s\n", report.Timestamp.Format("2006-01-02 15:04:05"))
    fmt.Printf("輸入檔案：%s\n", report.InputFile)
    fmt.Printf("輸入雜湊：%s\n", report.InputHash[:16]+"...")
    fmt.Println(strings.Repeat("-", 60))

    fmt.Println("\nGate 結果：")
    for _, gr := range report.GateResults {
        symbol := statusSymbol[gr.Status]
        fmt.Printf("\n%s [%s] %s (%dms) \n", symbol, gr.GateID, gr.GateName, gr.Duration)

        if len(gr.Issues) > 0 {
            fmt.Printf("   問題 (%d 項)：\n", len(gr.Issues))
            for _, issue := range gr.Issues {
                fmt.Printf("   [%s] %s：%s\n", issue.Severity, issue.ID, issue.Description)
            }
        }

        if len(gr.Warnings) > 0 {
            fmt.Printf("   警告 (%d 項)：\n", len(gr.Warnings))
            for _, warn := range gr.Warnings {
                fmt.Printf("   [WARN] %s：%s\n", warn.ID, warn.Description)
            }
        }
    }

    if len(report.RedFlags) > 0 {
        fmt.Printf("\n紅旗 (%d 項)：\n", len(report.RedFlags))
        for _, rf := range report.RedFlags {
            fmt.Printf("   [%s] %s：%s\n", rf.Severity, rf.Type, rf.Description)
        }
    }

    fmt.Println("\n" + strings.Repeat("-", 60))
    fmt.Printf("\n總體結論：%s %s\n", statusSymbol[report.OverallStatus], report.OverallStatus)
    fmt.Printf("追溯完整：%v | 證據充分：%v\n", report.TraceComplete, report.EvidenceSufficient)
    fmt.Println(strings.Repeat("=", 60))
}

func generateReportID() string {
    h := sha256.New()
    h.Write([]byte(time.Now().String()))
    return fmt.Sprintf("%x", h.Sum(nil))[:16]
}
```

```go
func computeFileHash(path string) string {
    data, err := os.ReadFile(path)
    if err != nil {
        return "unreadable"
    }
    h := sha256.Sum256(data)
    return fmt.Sprintf("%x", h)
}

// strings 套件補充（避免循環引入）
func strings_repeat(s string, n int) string {
    result := ""
    for i := 0; i < n; i++ {
        result += s
    }
    return result
}
```

---

7. CLI 入口

7.1 cmd/gate/main.go

```
package main

import (
    "fmt"
    "os"
    "strings"

    "github.com/spf13/cobra"
    "github.com/fatih/color"
    "github.com/yourorg/mc-ues-ci/internal"
    "github.com/yourorg/mc-ues-ci/internal/types"
)

var (
    termsFile      string
    gatesFile      string
    decisionsDir   string
    outputDir      string
    nonInteractive bool
    verbose        bool
)

var rootCmd = &cobra.Command{
    Use:   "mc-ues-gate",
    Short: "MC-UES CI/CD 驗證工具",
    Long: `Machine-Checkable Universal Engineering Science
CI/CD 驗證管道 - 約束 AI 輸出的語義治理工具`,
}

var checkCmd = &cobra.Command{
    Use:   "check [AI輸出檔案]",
    Short: "執行完整 Gate 驗證流程",
    Args:  cobra.ExactArgs(1),
    RunE: func(cmd *cobra.Command, args []string) error {
        inputFile := args[0]

        color.Blue("MC-UES Gate 驗證開始：%s\n", inputFile)

        // 載入設定
        terms, err := internal.LoadTermsConfig(termsFile)
        if err != nil {
            return fmt.Errorf("術語表載入失敗：%w", err)
        }
```

```
gatesConfig, err := internal.LoadGatesConfig(gatesFile)
if err != nil {
    return fmt.Errorf("Gate 設定載入失敗：%w", err)
}

// 載入 AI 輸出
output, err := internal.LoadAIOutput(inputFile)
if err != nil {
    return fmt.Errorf("AI 輸出載入失敗：%w", err)
}

// 載入決策日誌
decisions, err := internal.LoadAllDecisions(decisionsDir)
if err != nil {
    return fmt.Errorf("決策日誌載入失敗：%w", err)
}

// 執行 Gates
var gateResults []types.GateResult

// Gate 1
color.Yellow("執行 Gate 1：結構完整性...")
g1 := internal.Gate1Structure(output, gatesConfig.Gates.Gate1Structure)
gateResults = append(gateResults, g1)
printGateStatus(g1)

// Gate 2
color.Yellow("執行 Gate 2：術語邊界...")
g2 := internal.Gate2Terminology(output, terms, gatesConfig.Gates.Gate2Terminology)
gateResults = append(gateResults, g2)
printGateStatus(g2)

// Gate 3
color.Yellow("執行 Gate 3：紅旗偵測...")
g3 := internal.Gate3RedFlags(output, gatesConfig.Gates.Gate3RedFlags)
gateResults = append(gateResults, g3)
printGateStatus(g3)

// Gate 4
color.Yellow("執行 Gate 4：決策日誌一致性...")
g4 := internal.Gate4Decisions(output, decisions, gatesConfig.Gates.Gate4Decisions)
gateResults = append(gateResults, g4)
printGateStatus(g4)

// Gate 5
```

```
    color.Yellow("執行 Gate 5：人工核查點...")
    g5 := internal.Gate5Checkpoint(
        output,
        gateResults,
        decisions,
        gatesConfig.Gates.Gate5Checkpoint,
        nonInteractive,
    )
    gateResults = append(gateResults, g5)
    printGateStatus(g5)

    // 產生審計報告
    report := internal.GenerateAuditReport(inputFile, output, gateResults, gatesConfig)

    // 顯示報告
    if verbose {
        internal.PrintReport(report)
    }

    // 儲存報告
    reportPath, err := internal.SaveReport(report, outputDir)
    if err != nil {
        color.Red("報告儲存失敗：%v\n", err)
    } else {
        color.Cyan("審計報告已儲存：%s\n", reportPath)
    }

    // 決定退出碼
    switch report.OverallStatus {
    case types.StatusPass:
        color.Green("\n✅ MC-UES 驗證通過：完全符合\n")
        return nil
    case types.StatusConditionalPass:
        color.Yellow("\n⚠️  MC-UES 驗證條件通過：條件符合\n")
        return nil
    case types.StatusPendingHuman:
        color.Yellow("\n⏳ MC-UES 驗證等待人工核查\n")
        os.Exit(2)
    case types.StatusUnverified:
        color.Yellow("\n❓ MC-UES 驗證不可確認\n")
        os.Exit(3)
    default:
        color.Red("\n❌ MC-UES 驗證失敗：不符合\n")
        os.Exit(1)
    }
```

```
        return nil
    },
}

var distillCmd = &cobra.Command{
    Use:   "distill [審計報告] [輸出快照]",
    Short: "將審計報告蒸餾為狀態快照",
    Args:  cobra.ExactArgs(2),
    RunE: func(cmd *cobra.Command, args []string) error {
        reportFile := args[0]
        snapshotFile := args[1]

        color.Blue("執行蒸餾協議：%s → %s\n", reportFile, snapshotFile)

        snapshot, err := internal.DistillSnapshot(reportFile, snapshotFile)
        if err != nil {
            return fmt.Errorf("蒸餾失敗：%w", err)
        }

        color.Green("狀態快照已建立：%s（ID：%s）\n", snapshotFile, snapshot.ID)
        return nil
    },
}

var anchorCmd = &cobra.Command{
    Use:   "anchor",
    Short: "輸出語義錨點（供 AI 對話開始時注入）",
    RunE: func(cmd *cobra.Command, args []string) error {
        terms, err := internal.LoadTermsConfig(termsFile)
        if err != nil {
            return fmt.Errorf("術語表載入失敗：%w", err)
        }

        anchor := internal.GenerateSemanticAnchor(terms)
        fmt.Println(anchor)
        return nil
    },
}

var reportCmd = &cobra.Command{
    Use:   "report [審計報告檔案]",
    Short: "顯示審計報告",
    Args:  cobra.ExactArgs(1),
    RunE: func(cmd *cobra.Command, args []string) error {
        // 載入並顯示現有報告
        data, err := os.ReadFile(args[0])
```

```
        if err != nil {
            return fmt.Errorf("無法讀取報告：%w", err)
        }

        var report types.AuditReport
        if err := yaml.Unmarshal(data, &report); err != nil {
            return fmt.Errorf("報告格式錯誤：%w", err)
        }

        internal.PrintReport(&report)
        return nil
    },
}

func main() {
    rootCmd.PersistentFlags().StringVar(&termsFile, "terms", "config/terms.yaml", "術語表路徑")
    rootCmd.PersistentFlags().StringVar(&gatesFile, "gates", "config/gates.yaml", "Gate 設定路徑")
    rootCmd.PersistentFlags().StringVar(&decisionsDir, "decisions", "decisions/", "決策日誌目錄")
    rootCmd.PersistentFlags().StringVar(&outputDir, "output", "reports/", "報告輸出目錄")
    rootCmd.PersistentFlags().BoolVar(&nonInteractive, "non-interactive", false, "非互動模式（CI 環境使
        用）")
    rootCmd.PersistentFlags().BoolVar(&verbose, "verbose", false, "詳細輸出模式")

    rootCmd.AddCommand(checkCmd)
    rootCmd.AddCommand(distillCmd)
    rootCmd.AddCommand(anchorCmd)
    rootCmd.AddCommand(reportCmd)

    if err := rootCmd.Execute(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

func printGateStatus(gr types.GateResult) {
    symbols := map[types.GateStatus]string{
        types.StatusPass:            "✅",
        types.StatusFail:            "❌",
        types.StatusConditionalPass: "⚠️",
        types.StatusConditionalFail: "⚠️",
        types.StatusUnverified:      "❓",
        types.StatusPendingHuman:    "⏳",
    }
    symbol := symbols[gr.Status]
    fmt.Printf("  %s %s (%s) %dms\n", symbol, gr.GateName, gr.Status, gr.Duration)

    if len(gr.Issues) > 0 {
```

```
        for _, issue := range gr.Issues {
            fmt.Printf("     [%s] %s\n", issue.Severity, issue.Description)
        }
    }
}
```

8. VSCode 整合

8.1 `.vscode/tasks.json`

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "MC-UES: Check Current File",
      "type": "shell",
      "command": "./mc-ues-gate check ${file} --verbose",
      "group": {
        "kind": "test",
        "isDefault": true
      },
      "presentation": {
        "reveal": "always",
        "panel": "shared",
        "showReuseMessage": false
      },
      "problemMatcher": {
        "owner": "mc-ues",
        "fileLocation": ["relative", "${workspaceFolder}"],
        "pattern": {
          "regexp": "\\[HIGH\\]\\s(G\\d+-E\\d+)\\s(.+)",
          "message": 2
        }
      }
    },
    {
      "label": "MC-UES: Full Pipeline Check",
      "type": "shell",
      "command": "./mc-ues-gate check ${file} --verbose --non-interactive",
      "group": "test",
      "presentation": {
        "reveal": "always",
        "panel": "new"
      }
    },
    {
      "label": "MC-UES: Generate Semantic Anchor",
      "type": "shell",
      "command": "./mc-ues-gate anchor",
      "group": "none",
      "presentation": {
        "reveal": "always",
        "panel": "shared"
```

```
      }
    },
    {
      "label": "MC-UES: Distill Snapshot",
      "type": "shell",
      "command": "./mc-ues-gate distill reports/$(ls -t reports/ | head -1) snapshots/snapshot-$(date
        +%Y%m%d-%H%M%S).yaml",
      "group": "build",
      "presentation": {
        "reveal": "always",
        "panel": "shared"
      }
    },
    {
      "label": "MC-UES: Build Gate Tool",
      "type": "shell",
      "command": "go build -o mc-ues-gate ./cmd/gate/",
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "presentation": {
        "reveal": "silent",
        "panel": "shared"
      }
    },
    {
      "label": "MC-UES: Run All Tests",
      "type": "shell",
      "command": "go test ./... -v",
      "group": "test",
      "presentation": {
        "reveal": "always",
        "panel": "shared"
      }
    }
  ]
}
```

8.2 `.vscode/extensions.json`

```
{
  "recommendations": [
    "golang.go",
    "redhat.vscode-yaml",
    "ms-vscode.makefile-tools"
  ]
}
```

---

9. Git Hooks 整合

9.1 `hooks/pre-commit`

```
#!/bin/bash
# MC-UES pre-commit hook
# 自動對新增或修改的 AI 輸出檔案執行 Gate 驗證

set -e

# 顏色定義
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

echo -e "${YELLOW}[MC-UES] 執行 pre-commit Gate 驗證...${NC}"

# 取得暫存的 AI 輸出檔案（.yaml 在 ai-outputs/ 目錄下）
STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep '^ai-outputs/.*\.yaml$' || true)

if [ -z "$STAGED_FILES" ]; then
    echo -e "${GREEN}[MC-UES] 無 AI 輸出檔案需要驗證${NC}"
    exit 0
fi

# 確認 Gate 工具存在
if [ ! -f "./mc-ues-gate" ]; then
    echo -e "${YELLOW}[MC-UES] 正在編譯 Gate 工具...${NC}"
    go build -o mc-ues-gate ./cmd/gate/ || {
        echo -e "${RED}[MC-UES] Gate 工具編譯失敗${NC}"
        exit 1
    }
fi

# 對每個檔案執行驗證
FAILED=0
for FILE in $STAGED_FILES; do
    echo -e "${YELLOW}[MC-UES] 驗證：$FILE${NC}"

    if ./mc-ues-gate check "$FILE" --non-interactive; then
        echo -e "${GREEN}[MC-UES] ✅ 通過：$FILE${NC}"
    else
        EXIT_CODE=$?
        if [ $EXIT_CODE -eq 2 ]; then
            echo -e "${YELLOW}[MC-UES] ⏳ 等待人工核查：$FILE${NC}"
            echo -e "${YELLOW}[MC-UES] 請執行人工核查後再提交${NC}"
```

```
            FAILED=1
        else
            echo -e "${RED}[MC-UES] ❌ 驗證失敗：$FILE${NC}"
            FAILED=1
        fi
    fi
done

if [ $FAILED -ne 0 ]; then
    echo -e "${RED}[MC-UES] 提交被阻止：部分 AI 輸出未通過 MC-UES 驗證${NC}"
    echo -e "${YELLOW}[MC-UES] 請查看 reports/ 目錄中的審計報告${NC}"
    exit 1
fi

echo -e "${GREEN}[MC-UES] 所有 AI 輸出已通過驗證，允許提交${NC}"
exit 0
```

9.2 安裝 Hook 腳本 `Makefile`

```
.PHONY: build install-hooks test clean anchor check distill

# 編譯 Gate 工具
build:
    go build -o mc-ues-gate ./cmd/gate/
    @echo "✅ mc-ues-gate 編譯完成"

# 安裝 Git Hooks
install-hooks:
    cp hooks/pre-commit .git/hooks/pre-commit
    chmod +x .git/hooks/pre-commit
    @echo "✅ Git pre-commit hook 已安裝"

# 執行測試
test:
    go test ./... -v

# 清理
clean:
    rm -f mc-ues-gate
    rm -f reports/audit-*.yaml

# 產生語義錨點
anchor: build
    ./mc-ues-gate anchor

# 驗證指定檔案（用法：make check FILE=ai-outputs/output.yaml）
check: build
    ./mc-ues-gate check $(FILE) --verbose

# 非互動模式驗證（CI 環境）
check-ci: build
    ./mc-ues-gate check $(FILE) --non-interactive --verbose

# 蒸餾最新報告
distill: build
    ./mc-ues-gate distill \
        reports/$$(ls -t reports/ | head -1) \
        snapshots/snapshot-$$(date +%Y%m%d-%H%M%S).yaml

# 完整初始化
init: build install-hooks
    mkdir -p ai-outputs decisions snapshots reports
    @echo "✅ MC-UES CI/CD 環境初始化完成"
```

10. GitHub Actions CI

10.1 `.github/workflows/mc-ues-gate.yml`

```yaml
name: MC-UES Gate Verification

on:
  push:
    paths:
      - 'ai-outputs/**/*.yaml'
      - 'decisions/**/*.yaml'
  pull_request:
    paths:
      - 'ai-outputs/**/*.yaml'
      - 'decisions/**/*.yaml'

jobs:
  mc-ues-gate:
    name: MC-UES Gate Check
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.22'
          cache: true

      - name: Build Gate Tool
        run: |
          go build -o mc-ues-gate ./cmd/gate/
          echo "✅ mc-ues-gate compiled"

      - name: Run Gate Verification
        run: |
          FAILED=0
          for FILE in $(find ai-outputs/ -name "*.yaml" -newer decisions/ 2>/dev/null || find ai-outputs/ -name "*.yaml"); do
            echo "::group::Verifying $FILE"
            if ./mc-ues-gate check "$FILE" \
                --non-interactive \
                --verbose \
                --output reports/; then
              echo "✅ PASS: $FILE"
            else
```

```
          EXIT_CODE=$?
          if [ $EXIT_CODE -eq 2 ]; then
            echo "::warning::Pending human review: $FILE"
          else
            echo "::error::FAIL: $FILE"
            FAILED=1
          fi
        fi
        echo "::endgroup::"
      done
      exit $FAILED

  - name: Upload Audit Reports
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: mc-ues-audit-reports
      path: reports/
      retention-days: 90

  - name: Comment PR with Results
    if: github.event_name == 'pull_request' && always()
    uses: actions/github-script@v7
    with:
      script: |
        const fs = require('fs');
        const reports = fs.readdirSync('reports/').filter(f => f.endsWith('.yaml'));

        let summary = '## MC-UES Gate 驗證結果\n\n';

        for (const report of reports) {
          const content = fs.readFileSync(`reports/${report}`, 'utf8');
          // 解析 YAML 並格式化為摘要
          summary += `### ${report}\n\`\`\`\n${content.slice(0, 500)}...\n\`\`\`\n\n`;
        }

        github.rest.issues.createComment({
          issue_number: context.issue.number,
          owner: context.repo.owner,
          repo: context.repo.repo,
          body: summary
        });
```

11. 蒸餾協議

11.1 internal/distill.go

```go
package internal

import (
    "fmt"
    "os"
    "time"

    "gopkg.in/yaml.v3"
    "github.com/yourorg/mc-ues-ci/internal/types"
)

// DistillSnapshot 從審計報告蒸餾出狀態快照
// 蒸餾協議：任務完成後，將有效結論提取為可重用的基準點
func DistillSnapshot(reportFile, snapshotFile string) (*types.StateSnapshot, error) {
    // 載入審計報告
    data, err := os.ReadFile(reportFile)
    if err != nil {
        return nil, fmt.Errorf("無法讀取審計報告：%w", err)
    }

    var report types.AuditReport
    if err := yaml.Unmarshal(data, &report); err != nil {
        return nil, fmt.Errorf("審計報告格式錯誤：%w", err)
    }

    // 只有通過的報告可以蒸餾
    if report.OverallStatus == types.StatusFail {
        return nil, fmt.Errorf("不可蒸餾失敗的審計報告（狀態：%s）", report.OverallStatus)
    }

    // 建立快照
    snapshot := &types.StateSnapshot{
        ID:              generateSnapshotID(report),
        Timestamp:       time.Now(),
        DerivedFrom:     report.ReportID,
    }

    // 從報告中提取有效資訊
    snapshot.ValidArchitecture = extractValidArchitecture(report)
    snapshot.ActiveConstraints = extractActiveConstraints(report)
    snapshot.ExplicitlyExcluded = extractExcludedItems(report)
    snapshot.OpenProblems = extractOpenProblems(report)
```

```
    // 儲存快照
    snapshotData, err := yaml.Marshal(snapshot)
    if err != nil {
        return nil, fmt.Errorf("無法序列化快照：%w", err)
    }

    if err := os.WriteFile(snapshotFile, snapshotData, 0644); err != nil {
        return nil, fmt.Errorf("無法寫入快照：%w", err)
    }

    return snapshot, nil
}

func generateSnapshotID(report types.AuditReport) string {
    return fmt.Sprintf("snap-%s-%s",
        report.Timestamp.Format("20060102"),
        report.ReportID[:8])
}

func extractValidArchitecture(report types.AuditReport) string {
    // 從通過的 Gate 4 中提取有效架構描述
    for _, gr := range report.GateResults {
        if gr.GateID == "gate4" && gr.Status == types.StatusPass {
            return "通過決策日誌一致性驗證"
        }
    }
    return "部分通過，請參考審計報告"
}

func extractActiveConstraints(report types.AuditReport) []string {
    var constraints []string
    // 從紅旗中提取約束相關資訊
    for _, rf := range report.RedFlags {
        if rf.Type == types.RedFlagConstraintConflict {
            constraints = append(constraints,
                fmt.Sprintf("[衝突] %s", rf.Description))
        }
    }
    return constraints
}

func extractExcludedItems(report types.AuditReport) []string {
    var excluded []string
    for _, gr := range report.GateResults {
        for _, issue := range gr.Issues {
            if issue.Type == "提議排除方案" {
```

```
                excluded = append(excluded, issue.Description)
            }
        }
    }
    return excluded
}

func extractOpenProblems(report types.AuditReport) []string {
    var problems []string
    for _, gr := range report.GateResults {
        if gr.Status == types.StatusConditionalPass ||
            gr.Status == types.StatusConditionalFail {
            for _, issue := range gr.Issues {
                if issue.Severity == types.SeverityMedium {
                    problems = append(problems,
                        fmt.Sprintf("[%s] %s", gr.GateName, issue.Description))
                }
            }
        }
    }
    return problems
}

// GenerateSemanticAnchor 產生語義錨點（供 AI 對話開始時注入）
func GenerateSemanticAnchor(terms *types.TermsConfig) string {
    anchor := "# MC-UES 語義錨點\n"
    anchor += fmt.Sprintf("版本：%s | 更新：%s\n\n", terms.Version, terms.Updated)
    anchor += "## 本次對話適用術語定義\n\n"

    for termName, termDef := range terms.Terms {
        anchor += fmt.Sprintf("**%s** (%s)：%s\n",
            termName, termDef.EnglishAlias, termDef.Definition)
        if len(termDef.ForbiddenSynonyms) > 0 {
            anchor += fmt.Sprintf("  禁用同義詞：%v\n", termDef.ForbiddenSynonyms)
        }
        anchor += "\n"
    }

    anchor += "## 本次對話禁用詞\n\n"
    for _, fp := range terms.GlobalForbiddenPhrases {
        anchor += fmt.Sprintf("- 「%s」[%s]：%s\n", fp.Phrase, fp.Severity, fp.Reason)
    }

    anchor += "\n## 輸出格式要求\n\n"
    anchor += "所有輸出必須包含以下欄位：\n"
    anchor += "- intent_summary（意圖摘要）\n"
```

```
    anchor += "- assumptions（假設清單，不得為空）\n"
    anchor += "- uncertainty（不確定性聲明，不得為空）\n"
    anchor += "- source_markers（來源標記，每個結論必須標記）\n"
    anchor += "- predicate_results（判定結果）\n"
    anchor += "- red_flags（紅旗清單）\n"
    anchor += "- audit_conclusion（審計結論）\n"

    return anchor
}
```

---

12. 決策日誌系統

12.1 決策日誌完整範例 `decisions/ADR-002.yaml`

```yaml
id: "ADR-002"
title: "API 設計風格"
status: "confirmed"
version: "1.0"
created: "2024-01-02"
updated: "2024-01-02"
superseded_by: null

context: |
  專案需要定義統一的 API 設計規範，確保前後端介面一致性。

decision: |
  採用 RESTful API 設計，遵循 OpenAPI 3.0 規範。

rationale:
  - "業界廣泛採用，有成熟工具支援"
  - "團隊已有設計經驗"
  - "OpenAPI 規範支援自動文件生成"

constraints:
  - "所有端點必須有 OpenAPI 文件"
  - "回應格式統一使用 JSON"
  - "錯誤碼遵循 RFC 7807"
  - "版本號必須包含在 URL 路徑中（/v1/, /v2/）"

immutable_conditions:
  - "不得在未新增 ADR 的情況下引入 GraphQL 或 gRPC 作為主要 API 風格"
  - "不得廢棄 API 版本而不提供遷移指南"

excluded_alternatives:
  - option: "GraphQL"
    reason: "查詢複雜度過高，對當前團隊技術棧不必要"
  - option: "gRPC"
    reason: "瀏覽器相容性問題，不適合當前前端需求"

evidence:
  - type: "team_survey"
    description: "團隊技術棧調查"
    location: "docs/team-survey-2024.md"

tags:
  - "api"
```

```
  - "architecture"
  - "interface"
```

13. 狀態快照系統

13.1 狀態快照範例 `snapshots/snapshot-20240101-120000.yaml`

```yaml
id: "snap-20240101-abc12345"
timestamp: "2024-01-01T12:00:00+08:00"
derived_from: "abc12345def67890"

valid_architecture: "通過決策日誌一致性驗證（ADR-001, ADR-002）"

confirmed_decisions:
  - "ADR-001"
  - "ADR-002"

active_constraints:
  - "所有資料表必須有主鍵（ADR-001）"
  - "不得使用儲存程序作為業務邏輯載體（ADR-001）"
  - "所有端點必須有 OpenAPI 文件（ADR-002）"
  - "回應格式統一使用 JSON（ADR-002）"

open_problems:
  - "[術語邊界檢查] 部分模組文件使用了非標準術語"
  - "[決策日誌一致性] ADR-003 仍在 pending 狀態"

explicitly_excluded:
  - "MongoDB（已在 ADR-001 排除）"
  - "MySQL（已在 ADR-001 排除）"
  - "GraphQL（已在 ADR-002 排除）"
  - "gRPC（已在 ADR-002 排除）"

next_checkpoint: "模組介面定義完成後執行人工核查"
```

14. 術語表系統

14.1 AI 輸出格式範例 `ai-outputs/example-output.yaml`

這是 AI 應該產生的標準輸出格式，供 Gate 驗證：

```yaml
# AI 輸出範例 - 符合 MC-UES 規格
intent_summary: "設計使用者認證模組的架構"

ontology_mapping:
  需求: "使用者必須能夠透過電子郵件和密碼登入"
  約束: "密碼必須至少 12 字元，包含大小寫和數字"
  架構: "JWT-based 認證流程"

requirements:
  - "使用者可透過電子郵件和密碼進行認證"
  - "系統必須在 3 次錯誤嘗試後鎖定帳戶 15 分鐘"
  - "JWT Token 有效期為 24 小時"

constraints:
  - "密碼長度不少於 12 字元"
  - "密碼必須包含大寫字母、小寫字母和數字"
  - "不得以明文儲存密碼"
  - "Token 必須使用 RS256 演算法簽署"

architecture: |
  認證流程：
  1. 用戶提交電子郵件和密碼
  2. 系統查詢使用者資料庫
  3. 使用 bcrypt 驗證密碼雜湊
  4. 驗證成功後簽發 JWT Token
  5. Token 包含 userId, email, roles, exp

assumptions:
  - "假設使用者資料庫已存在且可連線"
  - "假設 RS256 私鑰已安全儲存在環境變數中"
  - "假設前端能夠安全儲存 JWT Token"

uncertainty: |
  - 帳戶鎖定後的解鎖機制（自動解鎖 vs 管理員手動解鎖）尚未確定
  - Refresh Token 是否需要實作待決策

source_markers:
  - claim: "JWT Token 有效期為 24 小時"
    source_type: "user_input"
    reference: "需求文件第 3.2 節"
    confidence: "high"
  - claim: "使用 bcrypt 驗證密碼雜湊"
    source_type: "inference"
    reference: "基於安全最佳實踐推論"
    confidence: "high"
  - claim: "3 次錯誤嘗試後鎖定帳戶"
```

```yaml
    source_type: "user_input"
    reference: "安全需求規格第 2 節"
    confidence: "high"

predicate_results:
  - predicate_name: "RequirementSatisfied"
    input: "使用者認證需求"
    result: true
    evidence:
      - "需求文件第 3.2 節"
      - "JWT 架構設計覆蓋所有認證場景"

  - predicate_name: "ConstraintSatisfied"
    input: "密碼安全約束"
    result: true
    evidence:
      - "bcrypt 雜湊保證密碼不以明文儲存"
      - "密碼長度約束在前端和後端雙重驗證"

  - predicate_name: "ArchitectureValid"
    input: "JWT 認證架構"
    result: true
    evidence:
      - "架構符合 ADR-002 API 設計規範"
      - "Token 使用 RS256，符合安全約束"

red_flags: []

audit_conclusion: |
  本輸出針對使用者認證模組架構設計。
  所有需求已明確定義且可驗證。
  主要不確定性（帳戶解鎖機制、Refresh Token）已明示。
  架構符合已確認的決策日誌（ADR-001, ADR-002）。
  建議：帳戶解鎖機制需要新增 ADR 進行決策。
```

---

15. 部署與使用說明

15.1 初始化專案

```
# 1. 克隆或初始化專案
git init mc-ues-project
cd mc-ues-project

# 2. 初始化 Go 模組
go mod init github.com/yourorg/mc-ues-ci

# 3. 安裝相依套件
go get gopkg.in/yaml.v3
go get github.com/spf13/cobra
go get github.com/fatih/color

# 4. 建立目錄結構
mkdir -p cmd/gate internal/types config decisions snapshots reports ai-outputs hooks .vscode
         .github/workflows

# 5. 複製所有程式碼到對應位置（依本文件各章節）

# 6. 執行初始化
make init
```

15.2 日常使用流程

```
# 驗證 AI 輸出
make check FILE=ai-outputs/my-output.yaml

# 取得語義錨點（在開始新的 AI 對話前執行）
make anchor

# 蒸餾最新報告為狀態快照
make distill

# CI 環境驗證（非互動模式）
make check-ci FILE=ai-outputs/my-output.yaml
```

15.3 AI 對話開始協議

每次開始新的 AI 對話前執行：

```
# 1. 取得語義錨點
./mc-ues-gate anchor > /tmp/semantic-anchor.txt

# 2. 取得最新狀態快照摘要
cat snapshots/$(ls -t snapshots/ | head -1)

# 3. 將以上內容注入 AI 對話的開頭
```

15.4 AI 輸出提交流程

```
# 1. 將 AI 輸出儲存為 YAML 格式
# 2. 放入 ai-outputs/ 目錄
# 3. Git add
git add ai-outputs/my-output.yaml

# 4. Git commit（pre-commit hook 自動執行 Gate 驗證）
git commit -m "feat: 認證模組架構設計"

# 如果驗證失敗，查看報告
cat reports/$(ls -t reports/ | head -1)
```

15.5 重置協議（偵測到漂移時）

```
# 1. 停止當前 AI 對話
# 2. 取得最近的有效快照
cat snapshots/$(ls -t snapshots/ | head -1)

# 3. 取得語義錨點
./mc-ues-gate anchor

# 4. 開始新的 AI 對話，注入：
#    - 語義錨點（術語定義）
#    - 最新狀態快照（有效架構 + 確認決策 + 排除項目）
#    - 當前任務的最小必要上下文
#    - 不注入：完整對話歷史、整個專案代碼
```

15.6 Gate 驗證失敗處理流程

```
Gate 驗證失敗
    ↓
查看失敗報告（reports/）
    ↓
識別失敗類型：
    ├── 結構問題（Gate 1）→ 修正 AI 輸出格式
    ├── 術語問題（Gate 2）→ 修正術語使用或更新術語表
    ├── 紅旗問題（Gate 3）→ 重新要求 AI 輸出，附上具體失敗項
    ├── 決策衝突（Gate 4）→ 確認是否需要新 ADR
    └── 人工核查（Gate 5）→ 人工審查後決定批准或拒絕
    ↓
修正後重新提交
```

附錄：MC-UES 判定式對應表

MC-UES 判定式	Gate 實作位置	驗證邏輯
RequirementSatisfied	Gate 1 + output 欄位	requirements 欄位非空且有來源標記
ConstraintSatisfied	Gate 4	與決策日誌約束無衝突
ArchitectureValid	Gate 4	符合已確認 ADR
EvidenceExists	Gate 1	source_markers 欄位非空
EvidenceSufficient	Gate 3	每個判定式結果有對應 evidence
TraceComplete	Gate 1	每個聲明有 source_marker
AuditPass	總體	所有 Gate 通過 ∧ 無 HIGH 紅旗
FullyConformant	總體	AuditPass ∧ TraceComplete ∧ EvidenceSufficient

本文件為 MC-UES CI/CD 完整架構規格。所有程式碼可直接用於實作，無需額外解釋。 版本：對應規格：MC-UES Machine-Checkable Universal Engineering Science